AI ENGINEER CASE STUDY · CUSTOMER JOURNEY ANALYTICS

Smart Inquiry Triage Assistant

Every customer inquiry classified, prioritised, routed and answered in about a second, with a confidence score that says when to ask a human instead.

`classify` + `retrieve` → `fuse` → `priority` → `route` → `notes`

PRESENTED BY

Swaraj Sonawane

STACK

LangGraph · Ollama · ChromaDB · Streamlit

RUNS

Fully local, no API key

CODE

github.com/swarajsonawane4/AI-Engineer-Case-Study

Today a person reads every message first

Inquiries arrive all day across the website, the app, the configurator and dealer systems. Before anyone can help, someone has to read each one and decide where it goes.

It is slow

The clock starts when the message arrives, not when a human finally opens it. That waiting time is invisible in most reporting.

It is inconsistent

Two people routing the same message send it to two different teams. The customer feels that as being passed around.

It does not scale

Volume grows with every new digital channel. Triage headcount does not grow with it.

What automated triage actually buys

Response time

Routing happens in about 1.3 seconds instead of waiting in an unread pile. The first human to touch the message is already the right human.

Consistency

The same inquiry gets the same category and the same queue every time, and the reasoning behind it is recorded.

Customer experience

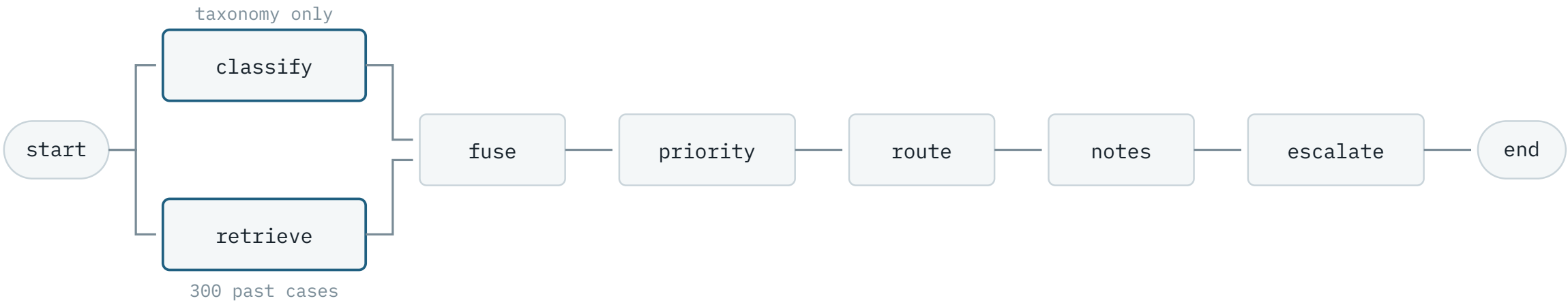
A brake fault never sits behind a brochure request. Urgency is assessed on arrival, not on opening.

One inquiry in, a full triage decision out

```
query: The brake warning light came on and the pedal feels soft. Is the car safe to drive?
category: service
priority: high
routed queue: Service Scheduling Team
confidence: 0.8628
resolution notes: Schedule an immediate appointment for the vehicle to inspect the brake
system and pedal feel. The agent should also review the maintenance history.
retrieved past cases: CASE-0003 · CASE-0051 · CASE-0188 · CASE-0262 · CASE-0180
```

The last line matters as much as the first. Every decision points back at the historical cases that produced it, so a reviewer can check the reasoning instead of trusting it.

Six steps, two of them in parallel



Classification and retrieval fan out from the start and join at the fusion step. Everything after that is sequential.

Why a graph and not a chain

Classifying and retrieving do not depend on each other. Both only need the raw text of the inquiry.

Running them as parallel branches saves a round trip. That is the small reason.

The real reason is independence

The classifier never sees the retrieved cases. So when the two agree, that agreement is real evidence.

If the classifier had been shown the neighbours first, it would tend to copy them, and their agreement would prove nothing. The whole confidence score is built on this.

A chain cannot express this. A chain has one order, so one step always sees the output of the last. Independence is a property of the shape, and the shape is the graph.

Routing is a lookup, not a prediction

In the historical data every category maps to exactly one queue, across all 300 rows. There is nothing here for a model to work out.

300 / 300	8	0 ms
rows where category determines queue	categories, 8 queues, one to one	added latency, and no token cost

So the routing table is built from the data at load time, and the loader raises an error if a category is ever seen mapping to two queues. A language model at this step could only introduce mistakes that a dictionary cannot make.

Priority comes from similar cases, not from the category

The category cannot decide urgency, because every category spans several priority levels. A billing question can be routine, or a duplicate charge worth thousands.

So priority is a similarity weighted vote over the retrieved cases, which carry the urgency signal the label does not.

> **The safety bias is deliberate**
>
> A very similar high priority neighbour lifts the result one level. Over-triaging wastes a few minutes. Under-triaging leaves a brake fault in a slow queue. Those costs are not equal, so the rule is not symmetric.

CATEGORY	LOW	MED	HIGH
service	20	18	17
technical	16	17	12
warranty	13	12	10
ordering	16	17	7
billing	14	18	8
configurator	22	13	0
general	26	4	0
other	19	1	0

Every operational category spans multiple priorities.

Confidence is measured, not asked for

SIGNAL	WEIGHT	WHAT IT CATCHES
Cross-check	0.40	Do the two independent predictors agree?
Agreement	0.35	How much retrieved weight backs the label?
Similarity	0.25	Are the neighbours close at all?

Each covers a failure the others miss. Similarity alone is high for a new inquiry that merely looks familiar. Agreement alone is high when retrieval is confidently wrong. The cross-check catches both but is only ever zero or one, so alone it cannot be thresholded.

What I deliberately did not use

Asking the model to rate its own confidence. A small model answers 0.9 to almost everything, because that number is generated text, not a measurement.

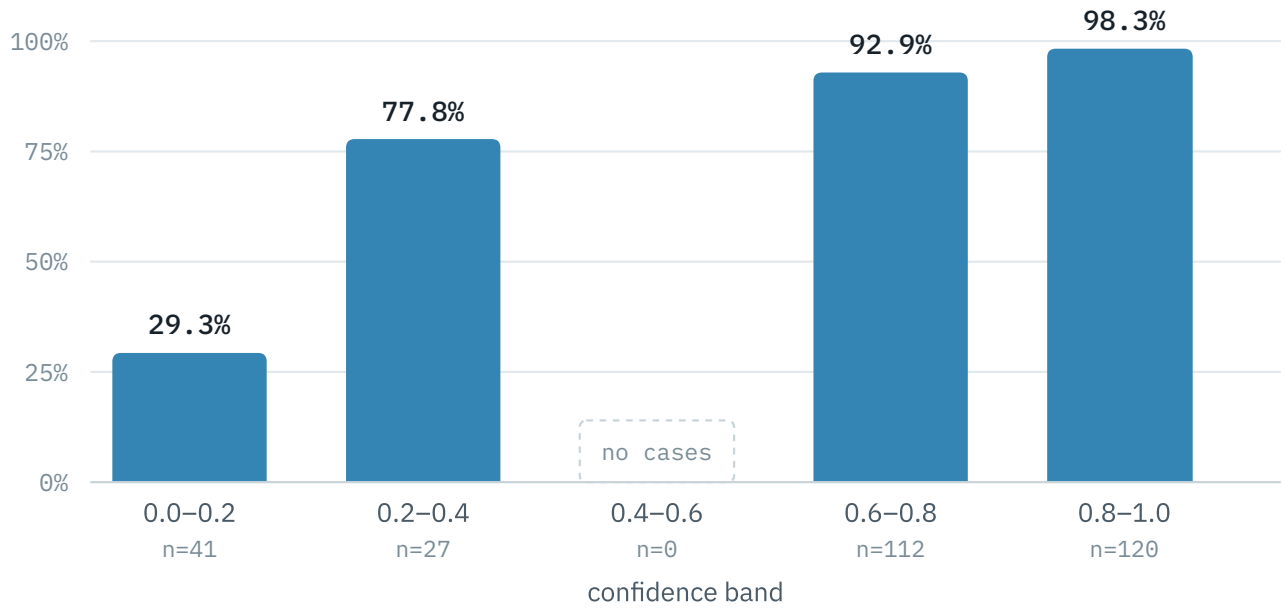
Every signal here is computed from something observable instead.

The honest trade-off

These weights are reasoned, not fitted. With 300 labelled cases they could be learned. The next slide is the measurement that would drive it.

Does the confidence score actually work?

If it does not track correctness it is decoration, because it decides what reaches a human. Measured by leave-one-out over all 300 cases.

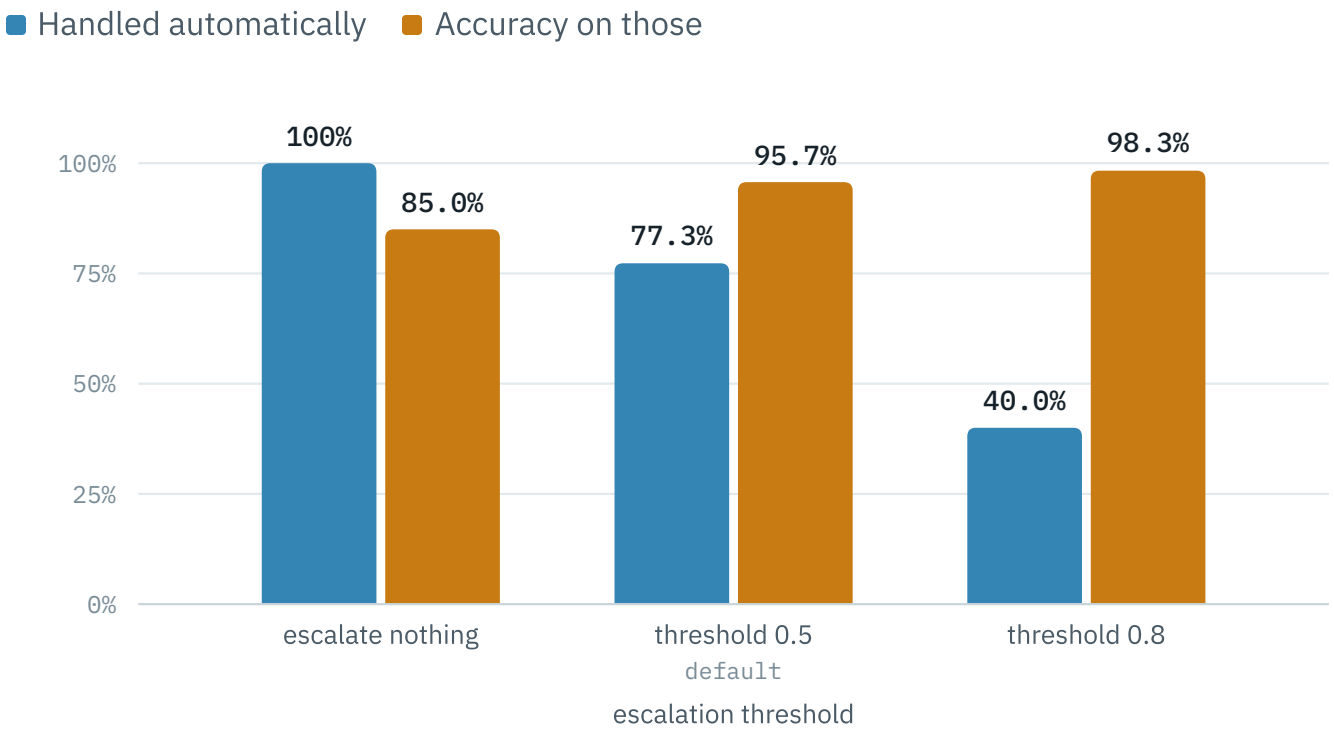


BAND	CASES	RIGHT	ACC.
0.0–0.2	41	12	29.3%
0.2–0.4	27	21	77.8%
0.4–0.6	0	—	—
0.6–0.8	112	104	92.9%
0.8–1.0	120	118	98.3%
All	**300**	**255**	**85.0%**

Accuracy rises with confidence, 29% in the lowest band to 98% in the highest. The bands sum back to the 85.0% headline. The empty middle band is the caveat: the cross-check signal is binary and carries the most weight, so scores cluster at the ends.

What that buys operationally

Because confidence tracks correctness, the escalation threshold becomes a real dial between coverage and accuracy.



THRESHOLD	AUTO	ACC.	TO HUMAN
0.0 none	300	85.0%	0
0.5 default	**232**	**95.7%**	**68**
0.8 strict	120	98.3%	180

At the default threshold, sending the least confident 68 inquiries to a person lifts accuracy on the remaining 232 from 85.0% to 95.7%.

Top-K was chosen by measuring, not by feel

Top-K is configurable, so the default should be defensible. Swept against ground truth over all 300 cases. Top-K only affects retrieval, so the sweep isolates it: the category column is the neighbour vote on its own, with no model involved. It is not the shipped pipeline, which reads 85.0%.

TOP-K	RETRIEVAL ONLY	PRIORITY ACC.	UNDER-TRIAGE
1	77.7%	52.7%	17.3%
3	80.7%	54.3%	16.7%
5	**83.7%**	**61.0%**	**15.0%**
7	83.3%	61.3%	13.7%
10	85.0%	61.3%	14.0%

Five is the knee

Accuracy climbs steeply to five and then flattens, while every extra neighbour adds latency and dilutes the vote. Going from three to five buys seven points of priority accuracy. Going from five to ten buys almost nothing.

Live demo

Three inquiries, chosen to show a different behaviour each time.

1 · The clear case

`"The brake warning light came on and the pedal feels soft. Is the car safe to drive?"`

High priority, service queue, confidence 0.86. Open the panel and show the five neighbours it reasoned from.

2 · The predictors disagree

`"The dealer charged me a diagnostic fee but said it was covered."`

The classifier reads it as billing, all five retrieved cases are service or warranty. The cross-check falls to zero, confidence lands at 0.19 and it escalates.

3 · Nothing close at all

`"What is a good recipe for banana bread with walnuts?"`

No neighbour is genuinely similar, so confidence collapses to 0.14 and it escalates instead of guessing.

What does not work yet

Priority is the weak step

61.0% exact agreement, and 95.7% within one level. Exact urgency is genuinely contested. Two human labellers would disagree on many of these too.

The knowledge base is small

300 cases. The `other` category has only 20, and it is the worst performing at 55.0%.

Notes are unevaluated

There is no ground truth for a generated note, so they are read for plausibility only. I am not claiming quality I did not measure.

Not production shaped

English only, single tenant, no authentication or rate limiting, and the weights are hand set rather than learned.

Where this goes next

Close the feedback loop

Every correction an agent makes becomes a new labelled case. The system improves without retraining anything. That is the main advantage of retrieval over a fine-tuned classifier, and it is the first thing I would build.

Learn the weights

Fit the confidence weights on held-out data and pick the threshold from the operating curve instead of the default.

Multilingual

Immediately necessary for a European customer base. The embedding model is the piece that changes.

Monitoring as a product signal

Track escalation rate and category drift. A sudden rise in one category is a product problem surfacing before it becomes a support cost.

In one slide

85.0%	95.7%	~1.3 s
category accuracy, all 300 cases	accuracy on the 77% handled automatically	per inquiry, fully local

Built as a graph on purpose

So classification and retrieval stay independent, which is what makes the confidence score mean something.

Measured, not asserted

Every constant in the config was set from an evaluation run, and one of them was wrong until that run caught it.

Knows when to stop

The system escalates rather than guessing, and shows the cases behind every decision.

Questions welcome.

`github.com/swarajsonawane4/AI-Engineer-Case-Study`